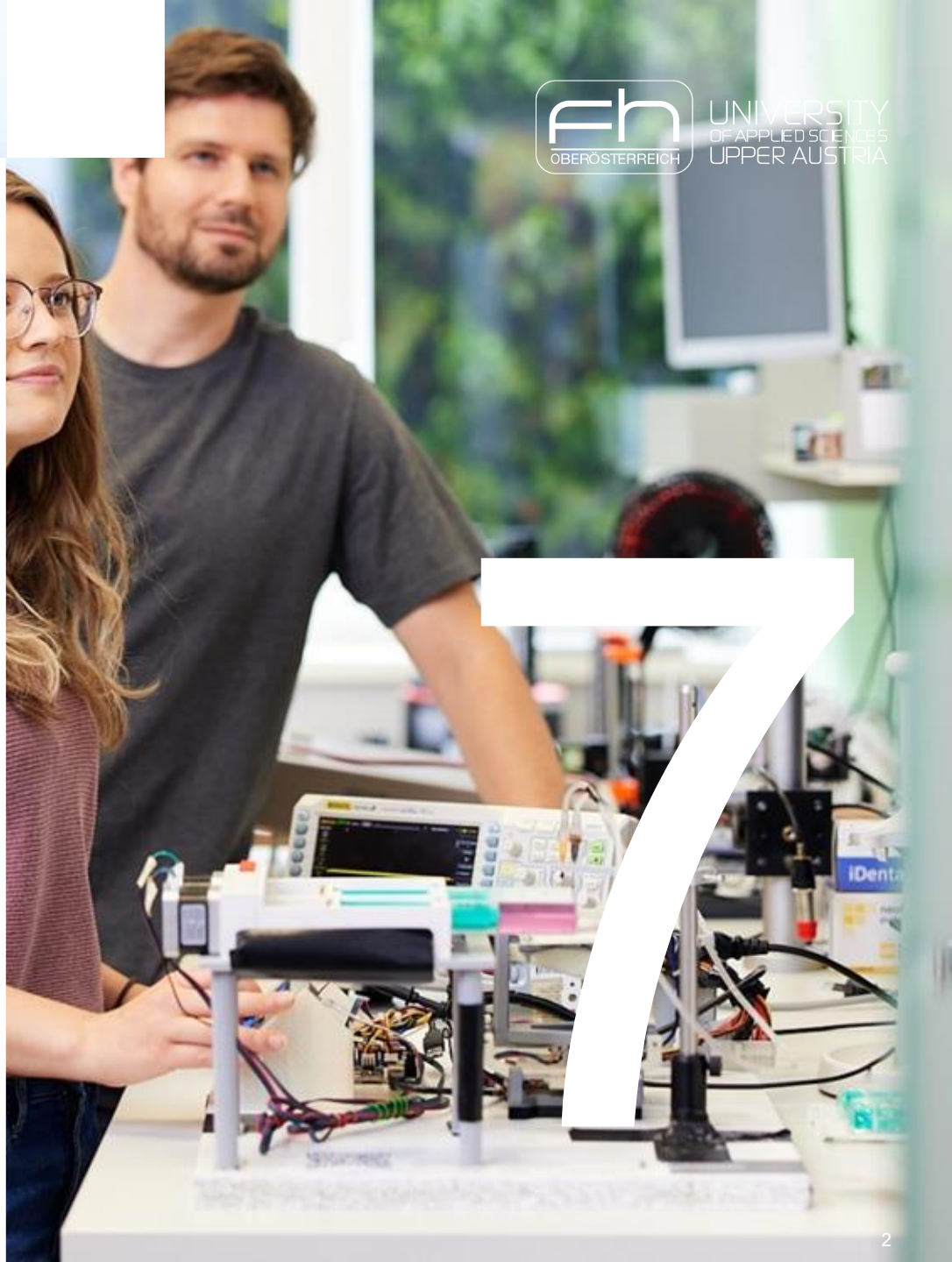
FH OBERÖSTERREICH
UNIVERSITY OF APPLIED SCIENCES UPPER AUSTRIA

# Game Engine Architectures

**Fachhochschule Oberösterreich**

# Game Engine Components



7

# Game Engine Components

## Game Loop

```
while (true)
{
    ms = duration_cast<milliseconds>(system_clock::now().time_since_epoch());
    milliseconds deltaTime = (ms - last);
    float deltaTimeSeconds = deltaTime.count() / 1000.0f;
    if (deltaTimeSeconds >= 1 / framerate)
    {
        deltaTimeSeconds = 1 / framerate;
        // Do something

        CheckUserInput();
        ApplyGamePhysics();
        Render();


        last = ms;
        framecounter++;
        cout << framecounter << endl;
    }
}
```

We would check if there is some user input
We apply the physical calculations to make the world dynamic
We render the output of the game

(for simplicity, we use the same update rate here)

# Game Engine Components

## Timer Calculations

The class “GameTimer” provides all necessary functions for controlling the frame rate

```
#pragma once
#include <chrono>
using namespace std::chrono;
class GameTimer
{
public:
    const float framerate = 10;
    milliseconds last;
    milliseconds ms;
    int framecounter = 0;
    float deltaTimeSeconds = 0;

    void initialize();
    bool checkFrame();
    void onFrameBegin();
    void onFrameEnd();
};
```

# Game Engine Components

## Timer Calculations

- Initialize: called to instantiate the timer objects
- checkFrame: determines if the main game loop is to be entered
- onFrameBegin: called at the beginning of each frame to update the timers
- onFrameEnd: called at the end of each frame

```
#pragma once
#include <chrono>
using namespace std::chrono;
class GameTimer
{
public:
    const float framerate = 10;
    milliseconds last;
    milliseconds ms;
    int framecounter = 0;
    float deltaTimeSeconds = 0;

    void initialize();
    bool checkFrame();
    void onFrameBegin();
    void onFrameEnd();
};
```

# Game Engine Components

## Timer Calculations

```
void GameTimer::initialize()
{
    last = duration_cast<milliseconds>(system_clock::now().time_since_epoch());
    ms = duration_cast<milliseconds>(system_clock::now().time_since_epoch());
    framecounter = 0;
}

bool GameTimer::checkFrame()
{
    ms = duration_cast<milliseconds>(system_clock::now().time_since_epoch());
    milliseconds deltaTime = (ms - last);
    deltaTimeSeconds = deltaTime.count() / 1000.0f;
    return deltaTimeSeconds >= 1 / framerate;
}

void GameTimer::onFrameBegin()
{
    deltaTimeSeconds = 1 / framerate;
}

void GameTimer::onFrameEnd()
{
    last = ms;
    framecounter++;
}
```

# Game Engine Components

## Game Loop

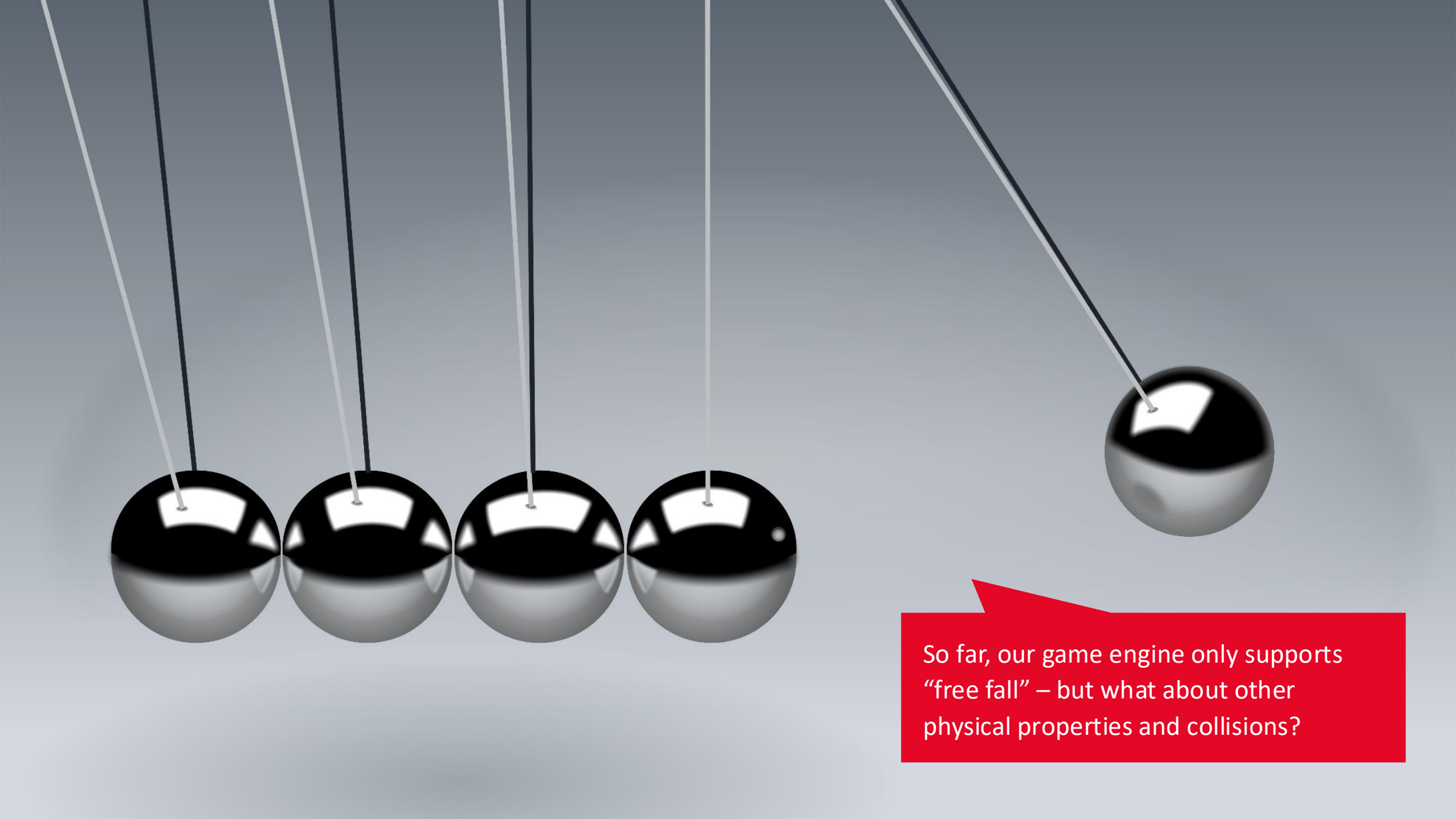
The *GameTimer* helps us to improve our game loop

```
int main()
{
    GameTimer timer;
    timer.initialize();
    while (true)
    {
        if (timer.checkFrame())
        {
            timer.onFrameBegin();

            // Do something

            timer.onFrameEnd();
        }

    }
}
```



So far, our game engine only supports “free fall” – but what about other physical properties and collisions?

# Game Engine Components

## Physical Properties

- We want to support
  - **Rigidbodies:** whether an object is included in the physical dynamics simulation
  - **Colliders:** whether an object is included in the collision detection
  - **Bumping:** whether an object will bounce after colliding

# Game Engine Components

## Physics Loop

```
class PhysicsObject {
public:
    char const* name;
    float x = 0;
    float y = 0;
    float width = 0;
    float height = 0;
    float velocity = 0;
    bool collider = 0;
    bool isRigidBody = 1;
    bool bump = 0;
    float bumpFactor = 0.75f;
    PhysicsObject();
    PhysicsObject(float x, float altitude, float width, float height);
    void UpdatePhysics(float deltaTime);
    bool IsCollidingWith(PhysicsObject& o);
    virtual void Update(float deltaTime);
    virtual char getVisual();
```

Objects now have a height/width
(before, they were all just 1 unit in size,
important for collisions and rendering)

# Game Engine Components

## Physics Loop

```
class PhysicsObject {
public:
    char const* name;
    float x = 0;
    float y = 0;
    float width = 0;
    float height = 0;
    float velocity = 0;
    bool collider = 0;
    bool isRigidBody = 1;
    bool bump = 0;
    float bumpFactor = 0.75f;
    PhysicsObject();
    PhysicsObject(float x, float altitude, float width, float height);
    void UpdatePhysics(float deltaTime);
    bool IsCollidingWith(PhysicsObject& o);
    virtual void Update(float deltaTime);
    virtual char getVisual();
```

*PhysicsObjects* can now be configured to not fall (i.e., isRigidBody [true|false])

And it they should bump when colliding with the floor
(with bumpFactor*-velocity)

# Game Engine Components

## Physics Loop

```
class PhysicsObject {
public:
    char const* name;
    float x = 0;
    float y = 0;
    float width = 0;
    float height = 0;
    float velocity = 0;
    bool collider = 0;
    bool isRigidBody = 1;
    bool bump = 0;
    float bumpFactor = 0.75f;
    PhysicsObject();
    PhysicsObject(float x, float altitude, float width, float height);
    void UpdatePhysics(float deltaTime);
    bool IsCollidingWith(PhysicsObject& o);
    virtual void Update(float deltaTime);
    virtual char getVisual();
```

We implement a simple collision system. *IsCollidingWith* checks if this is in collision with another *PhysicsObject* o

# Game Engine Components

## Physics Loop

```
class PhysicsObject {
public:
    char const* name;
    float x = 0;
    float y = 0;
    float width = 0;
    float height = 0;
    float velocity = 0;
    bool collider = 0;
    bool isRigidBody = 1;
    bool bump = 0;
    float bumpFactor = 0.75f;
    PhysicsObject();
    PhysicsObject(float x, float altitude, float width, float height);
    void UpdatePhysics(float deltaTime);
    bool IsCollidingWith(PhysicsObject& o);
    virtual void Update(float deltaTime);
    virtual char getVisual();
```

We add a virtual *Update* function to be overriden from child actors, so that user can use our engine “just as any other” engine

# Game Engine Components

## Physics Loop

```
void PhysicsObject::UpdatePhysics(float deltaTime)
{
    if (!isRigidBody) return;

    float newHeight = this->velocity * deltaTime +
        (this->y) - 0.5f * Gravity * (deltaTime * deltaTime);
    float newVelocity = this->velocity - Gravity * deltaTime;
    this->y = newHeight;
    this->velocity = newVelocity;
    if (this->collider && this->y < 0)
    {
        this->y = 0.0f;
        if (bump)
            this->velocity = -(this->velocity * bumpFactor);
    }
}
```

General *UpdatePhysics*:
we just return when no rigid body is defined

# Game Engine Components

## Physics Loop

```
void PhysicsObject::UpdatePhysics(float deltaTime)
{
    if (!isRigidBody) return;

    float newHeight = this->velocity * deltaTime +
        (this->y) - 0.5f * Gravity * (deltaTime * deltaTime);
    float newVelocity = this->velocity - Gravity * deltaTime;
    this->y = newHeight;
    this->velocity = newVelocity;
    if (this->collider && this->y < 0)
    {
        this->y = 0.0f;
        if (bump)
            this->velocity = -(this->velocity * bumpFactor);
    }
}
```

When we collide with the floor, we reset negative velocity*bumpFactor if bump is true

What happens if bumpFactor is ==1?
We oscillate forever

# Game Engine Components

## Physics Loop

```
bool PhysicsObject::IsCollidingWith(PhysicsObject& o)
{
    bool collideX = false;
    bool collideY = false;
    for (float i = this->x; i <= (this->x + this->width); i += 0.01f)
    {
        if (i >= o.x && i <= (o.x + o.width))
        {
            collideX = true;
            break;
        }
    }
    for (float i = this->y; i <= (this->y + this->height); i += 0.01f)
    {
        if (i >= o.y && i <= (o.y + o.height))
        {
            collideY = true;
            break;
        }
    }
    return collideX && collideY;

}
```

*IsCollidingWidth* Method:
We check for both axis x/y if the reference object o's x/y is somewhere in our position/size range

What collision method is this?
Seperating Axis Theorem

What could be improved?
Iterate also through o to increase precision!



So far, we did not include a world/
collision/physics model
(our instances were in an array with fixed size)

# Game Engine Components

## Game World

```
#include "PhysicsObject.h"
#include <vector>
using namespace std;

class PhysicsObject;

class WorldModel {
private:
    vector<PhysicsObject*> worldModel;
public:
    WorldModel();
    void addObject(PhysicsObject &o);
    void removeObject(PhysicsObject &o);
    ~WorldModel();
    vector<PhysicsObject*> getObjects();
    int getSize();
    void Step(float deltaTime);
    static WorldModel* GetSingleton();
};
```

We implement a *WorldModel* class

It contains a vector of pointers to *PhysicObjects*

A vector is a C++ “ready to go” collection similar to a variable-sized array (in particular, a list!)

# Game Engine Components

## Game World

```
#include "PhysicsObject.h"
#include <vector>
using namespace std;

class PhysicsObject;

class WorldModel {
private:
    vector<PhysicsObject*> worldModel;
public:
    WorldModel();
    void addObject(PhysicsObject &o);
    void removeObject(PhysicsObject &o);
    ~WorldModel();
    vector<PhysicsObject*> getObjects();
    int getSize();
    void Step(float deltaTime);
    static WorldModel* GetSingleton();
};
```

We define a couple of methods for our WorldModel, to

- Add/remove PhysicsObjects
- Get a list of all objects
- A “step” function to be called once per frame
- A Singleton to allow sharing the world model across code in the engine

# Game Engine Components

## Game World Model Singleton

```
static WorldModel* global;
WorldModel::WorldModel()
{
    worldModel;
    global = this;
}

WorldModel* WorldModel::GetSingleton()
{
    return global;
}
```

Therefore, we must allow a GameObject (like *PhysicsObject* or *Character*) to access the *WorldModel*

As we have only one *WorldModel*, we provide a static instance to others

What is "*static*"?
Functionality that exists once per class, independent of any instances

# Game Engine Components

## Game World Model Singleton

```
static WorldModel* global;
WorldModel::WorldModel()
{
    worldModel;
    global = this;
}

WorldModel* WorldModel::GetSingleton()
{
    return global;
}
```

We define a static WorldModel object in the WorldModel class itself, and whenever we initialize it, we set it to the current instance

# Game Engine Components

## Game World Model Singleton

```
static WorldModel* global;
WorldModel::WorldModel()
{
    worldModel;
    global = this;
}

WorldModel* WorldModel::GetSingleton()
{
    return global;
}
```

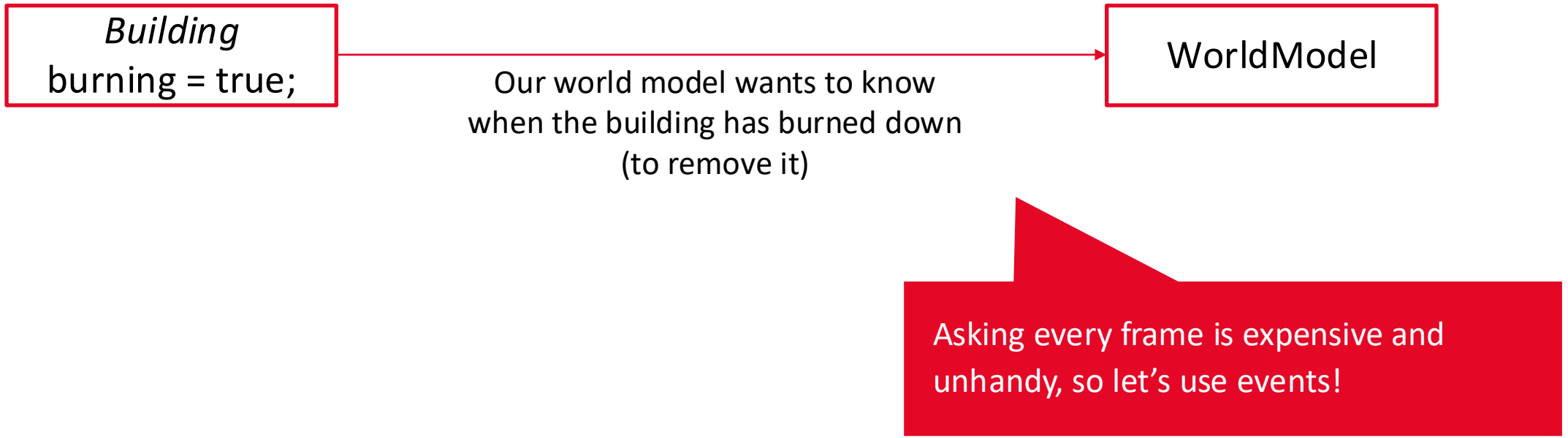
Consequently, any other class can call the static *WorldModel* with the *GetSingleton()* function



When collisions occur, we want to throw an EVENT

# Game Engine Components

## Event Handling



# Game Engine Components

## Event Handling

*Building*
burning = true;
event
OnBurnedDown

WorldModel
OnBurnedHandler

The Building class can throw an event when it has burned down

# Game Engine Components

## Event Handling

*Building*
burning = true;
event
OnBurnedDown

WorldModel
OnBurnedHandler

```
void OnBurnedHandler(object* source) {
   this->worldModel.Remove(*source);
}
```

The WorldModel class implements a function that should be called when a building has burned down so that it can be removed

# Game Engine Components

## Event Handling

*Building*
burning = true;
event
OnBurnedDown

WorldModel
OnBurnedHandler

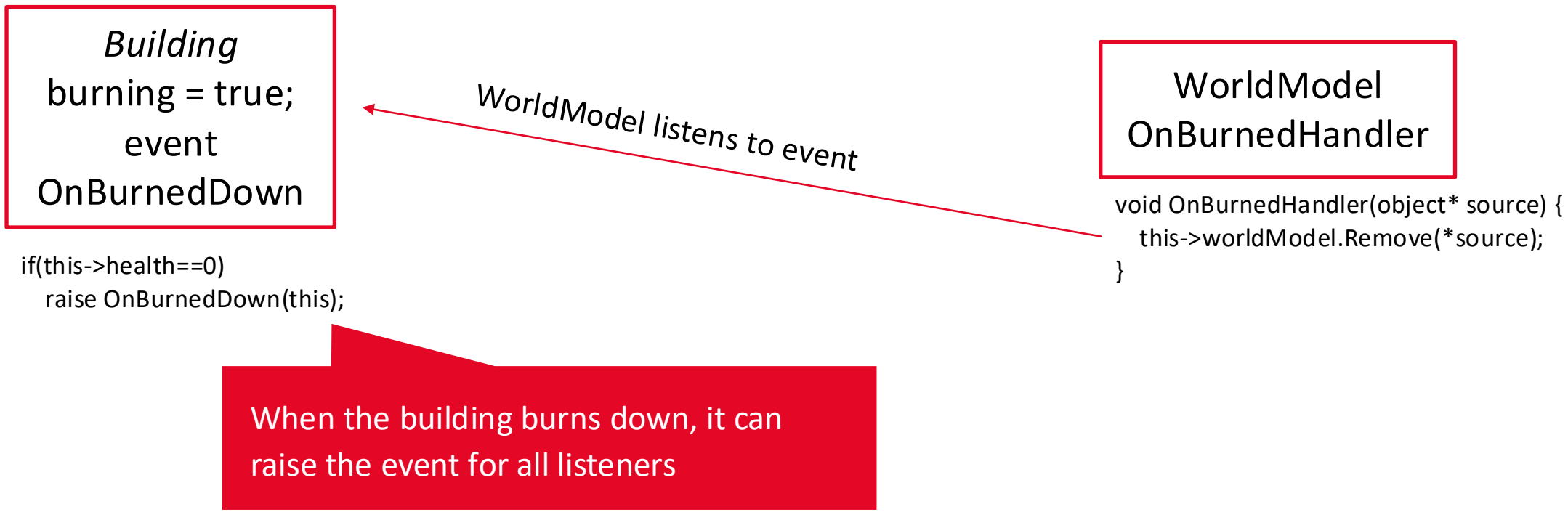
```
void OnBurnedHandler(object* source) {
   this->worldModel.Remove(*source);
}
```

We must “hook” this method to a building instance‘s OnBurnedDown event, i.e., we “listen” for the event

- The event can be hooked to multiple buildings
- Multiple classes can hook to the event – not just the WorldModel (for example, a progress bar)

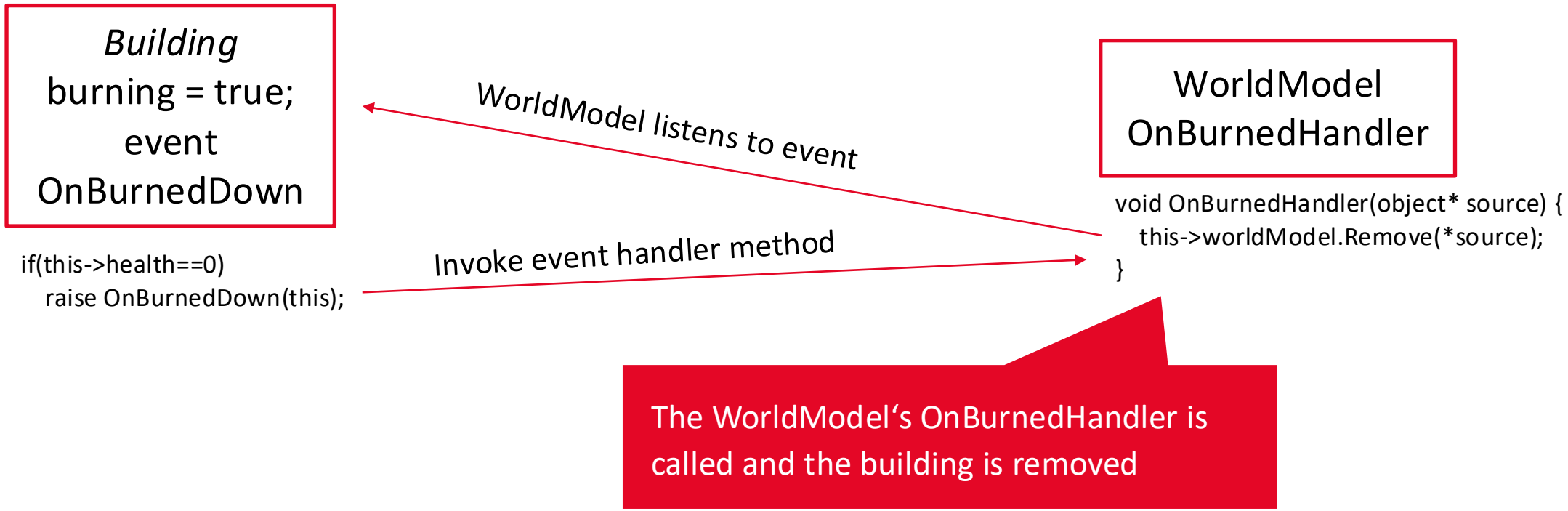
# Game Engine Components

## Event Handling



# Game Engine Components

## Event Handling





Let’s apply events to our collision system!

# Game Engine Components

## Collision Events

```
class PhysicsObject {
public:
    __event void OnCollisionEvent(PhysicsObject *sender, PhysicsObject* receiver);
    void hookEvent(PhysicsObject* pSource);
    void unhookEvent(PhysicsObject* pSource);
    virtual void OnCollisionHandler(PhysicsObject* sender, PhysicsObject* receiver);

};
```

First, we define the event using the __event keyword

Each event has a sender (i.e., who raises the event)
We add another parameter receiver – both sender/receiver are 2 *PhysicsObjects* colliding

We do not implement the event; this is done automatically by the compiler!

# Game Engine Components

## Collision Events

```
class PhysicsObject {
public:
    __event void OnCollisionEvent(PhysicsObject *sender, PhysicsObject* receiver);
    void hookEvent(PhysicsObject* pSource);
    void unhookEvent(PhysicsObject* pSource);
    virtual void On  llisionHandler(PhysicsObject* sender, PhysicsObject* receiver);

};
```

We define 2 methods so that

- Another *PhysicsObject* can listen to the event (hookEvent)
- Another *PhysicsObject* can stop listening to the event (unhookEvent)

# Game Engine Components

## Collision Events

```
class PhysicsObject {
public:
    __event void OnCollisionEvent(PhysicsObject *sender, PhysicsObject* receiver);
    void hookEvent(PhysicsObject* pSource);
    void unhookEvent(PhysicsObject* pSource);
    virtual void OnCollisionHandler(PhysicsObject* sender, PhysicsObject* receiver);

};
```

We define the *Handler* method, which is called when sender is colliding with a receiver
Here, *PhysicsObject* is **both sender and receiver** of the event!

# Game Engine Components

## Collision Events

```
void PhysicsObject::hookEvent(PhysicsObject* pSource)
{
    __hook(&PhysicsObject::OnCollisionEvent, pSource, &PhysicsObject::OnCollisionHandler);
}
```

Register a *__hook* (i.e., listener) for the event:
We tell the compiler, that *pSource* wants to listen to the *OnCollisionEvent*, and if it is raised, the *OnCollisionHandler* method should be called

# Game Engine Components

## Collision Events

```
void PhysicsObject::unhookEvent(PhysicsObject* pSource)
{
    __unhook(&PhysicsObject::OnCollisionEvent, pSource, &PhysicsObject::OnCollisionHandler);
}
```

The inverse happens when we *__unhook* (i.e., stop listening)
We tell the compiler, that *pSource* should stop invoking *OnCollisionHandler* when the event is raised using the *__unhook* keyword

Does the event system work now?
No – no events are registered or raised yet!

# Game Engine Components

## Collision Events

```
void PhysicsObject::OnCollisionHandler(PhysicsObject* sender, PhysicsObject* receiver)
{
    std::cout << sender->name << " colliding with " << receiver->name << std::endl;
}
```

Currently, the *Handler* method that is called when the event is raised does nothing but telling us on the command line, which objects collided

# Game Engine Components

## Game World

```
void WorldModel::addObject(PhysicsObject& o)
{
    worldModel.push_back(&o);
    if (o.collider)
    {
        for (PhysicsObject* i : worldModel)
        {
            if (i->collider && &o!=i)
            {
                o.hookEvent(i);
                i->hookEvent(&o);
            }
        }
    }
}
```

Whenever we add a new *PhysicsObject* to our *WorldModel* class…

# Game Engine Components

## Game World

```
void WorldModel::addObject(PhysicsObject& o)
{
    worldModel.push_back(&o);
    if (o.collider)
    {
        for (PhysicsObject* i : worldModel)
        {
            if (i->collider && &o!=i)
            {
                o.hookEvent(i);
                i->hookEvent(&o);
            }
        }
    }
}
```

Whenever we add a new *PhysicsObject* to our *WorldModel* class…
… we append this object to the collection *vector* (at the end using the *push_back* method)

# Game Engine Components

## Game World

```
void WorldModel::addObject(PhysicsObject& o)
{
    worldModel.push_back(&o);
    if (o.collider)
    {
        for (PhysicsObject* i : worldModel)
        {
            if (i->collider && &o!=i)
            {
                o.hookEvent(i);
                i->hookEvent(&o);
            }
        }
    }
}
```

If the newly added *PhysicsObject* is a collider, we iterate through all other *PhysicsObjects* that are already in the vector

# Game Engine Components

## Game World

```
void WorldModel::addObject(PhysicsObject& o)
{
    worldModel.push_back(&o);
    if (o.collider)
    {
        for (PhysicsObject* i : worldModel)
        {
            if (i->collider && &o!=i)
            {
                o.hookEvent(i);
                i->hookEvent(&o);
            }
        }
    }
}
```

If both are *colliders*, we register the events on both sides (so that both can make use of the collision when raised)

# Game Engine Components

## Game World Step Function

```
void WorldModel::Step(float deltaTime)
{
    for (PhysicsObject* o : worldModel)
    {
        o->UpdatePhysics(deltaTime);

        for (PhysicsObject* i : worldModel)
        {
            if (i != o && i->collider&&o->collider)
            {
                bool collision = i->IsCollidingWith(*o);
                if (collision)
                {
                    __raise o->OnCollisionEvent(o,i);
                    __raise i->OnCollisionEvent(i,o);
                }
            }
        }
        o->Update(deltaTime);
    }
}
```

The *Step* function on the WorldModel is called once per frame

# Game Engine Components

## Game World Step Function

```
void WorldModel::Step(float deltaTime)
{
    for (PhysicsObject* o : worldModel)
    {
        o->UpdatePhysics(deltaTime);

        for (PhysicsObject* i : worldModel)
        {
            if (i != o && i->collider&&o->collider)
            {
                bool collision = i->IsCollidingWith(*o);
                if (collision)
                {
                    __raise o->OnCollisionEvent(o,i);
                    __raise i->OnCollisionEvent(i,o);
                }
            }
        }
        o->Update(deltaTime);
    }
}
```

We loop over all *PhysicsObjects* in the game, and apply the *UpdatePhysics* method (i.e., objects fall down or bump)

# Game Engine Components

## Game World Step Function

```
void WorldModel::Step(float deltaTime)
{
    for (PhysicsObject* o : worldModel)
    {
        o->UpdatePhysics(deltaTime);

        for (PhysicsObject* i : worldModel)
        {
            if (i != o && i->collider&&o->collider)
            {
                bool collision = i->IsCollidingWith(*o);
                if (collision)
                {
                    __raise o->OnCollisionEvent(o,i);
                    __raise i->OnCollisionEvent(i,o);
                }
            }
        }
        o->Update(deltaTime);
    }
}
```

We evaluate for each object if it would collide with any other object (second loop)

Preconditions: both objects are a collider and i!=o (we do not want to collide with itself)

# Game Engine Components

## Game World Step Function

```
void WorldModel::Step(float deltaTime)
{
    for (PhysicsObject* o : worldModel)
    {
        o->UpdatePhysics(deltaTime);

        for (PhysicsObject* i : worldModel)
        {
            if (i != o && i->collider&&o->collider)
            {
                bool collision = i->IsCollidingWith(*o);
                if (collision)
                {
                    __raise o->OnCollisionEvent(o,i);
                    __raise i->OnCollisionEvent(i,o);
                }
            }
        }
        o->Update(deltaTime);
    }
}
```

If a collision happens, we raise the *OnCollisionEvent* on both sides (i.e, each PhysicsObject is once sender and once receiver of the collision)

# Game Engine Components

## Game World Step Function

```
void WorldModel::Step(float deltaTime)
{
    for (PhysicsObject* o : worldModel)
    {
        o->UpdatePhysics(deltaTime);

        for (PhysicsObject* i : worldModel)
        {
            if (i != o && i->collider&&o->collider)
            {
                bool collision = i->IsCollidingWith(*o);
                if (collision)
                {
                    __raise o->OnCollisionEvent(o,i);
                    __raise i->OnCollisionEvent(i,o);
                }
            }
        }
        o->Update(deltaTime);
    }
}
```

At the end, we execute any *PhysicsObject*'s default *Update* method

# Game Engine Components

## Game Loop

```
CheckUserInput();
ApplyGamePhysics();
Render();
```

The physics engine of the (simple) game engine is completed…



What about characters and input?

# Game Engine Components

## Game Engine Characters

```
class Character : public PhysicsObject{
public:
    Character();
    void MoveLeft();
    void MoveRight();
    void Jump();
    char getVisual() override;
    void Update(float deltaTime) override;
    void OnCollisionHandler(PhysicsObject* sender, PhysicsObject* receiver) override;
};
```

We create a Character class that inherits *PhysicsObject*

It provides functions for movement and has its own visual

# Game Engine Components

## Game Engine Characters

```
class Character : public PhysicsObject{
public:
    Character();
    void MoveLeft();
    void MoveRight();
    void Jump();
    char getVisual() override;
    void Update(float deltaTime) override;
    void OnCollisionHandler(PhysicsObject* sender, PhysicsObject* receiver) override;
};
```

Just as in most Game Engines, we can now override and use functionality from the Engine's base classes

Our character can implement the *Update* and *OnCollisionHandler* methods

Let us delete other objects when colliding with the character!

# Game Engine Components

## Game Character

In the constructor, we call the base constructor and activate the collider – our character should not fall through the floor

```
Character::Character():PhysicsObject(0,0)
{
    this->collider = true;
}

void Character::MoveLeft()
{
    this->xPos--;
}

void Character::MoveRight()
{
    this->xPos++;
}

void Character::Jump()
{
    if(this->altitude<=0)
    this->velocity = 10;
}

char Character::getVisual()
{
    return 'X';
}
```

# Game Engine Components

## Game Character

For movement, we just control the x-position of the underlying *PhysicsObject*

```
Character::Character():PhysicsObject(0,0)
{
    this->collider = true;
}

void Character::MoveLeft()
{
    this->xPos--;
}

void Character::MoveRight()
{
    this->xPos++;
}

void Character::Jump()
{
    if(this->altitude<=0)
    this->velocity = 10;
}

char Character::getVisual()
{
    return 'X';
}
```

# Game Engine Components

## Game Character

```
Character::Character():PhysicsObject(0,0)
{
    this->collider = true;
}

void Character::MoveLeft()
{
    this->xPos--;
}

void Character::MoveRight()
{
    this->xPos++;
}

void Character::Jump()
{
    if(this->altitude<=0)
    this->velocity = 10;
}

char Character::getVisual()
{
    return 'X';
}
```

For a jump, we give it some positive velocity whenever the character is on the ground

# Game Engine Components

## Game Character OnCollisionHandler

```
void Character::OnCollisionHandler(PhysicsObject* sender, PhysicsObject* receiver)
{
    //std::cout << sender->name << " colliding with " << receiver->name << std::endl;
    WorldModel::GetSingleton()->removeObject(*sender);
}
```

Due to polymorphism, the derived classes' *OnCollisionHandler* will be called. Here, we tell the static *WorldModel* to remove the sender object (i.e., the *PhysicsObject* our *character* is colliding with) from the Game World.




Input System

# Game Engine Components

## Input

Our input class abstracts the specific keyboard layout and provides 4 main directions

```
class Input {
private:
    static const int Up = 'W';
    static const int Down = 'S';
    static const int Left = 'A';
    static const int Right = 'D';

public:
    bool upPressed = false;
    bool downPressed = false;
    bool leftPressed = false;
    bool rightPressed = false;

    void GetInput();
};
```

# Game Engine Components

## Input

```
class Input {
private:
    static const int Up = 'W';
    static const int Down = 'S';
    static const int Left = 'A';
    static const int Right = 'D';

public:
    bool upPressed = false;
    bool downPressed = false;
    bool leftPressed = false;
    bool rightPressed = false;

    void GetInput();
};
```

The input object can be queried if particular actions are active

# Game Engine Components

## Input

We include Windows.h to access the *GetKeyState* function

```
#include "Input.h"
#include <Windows.h>
void Input::GetInput()
{
    if (GetKeyState(Input::Up) & 0x8000)
        this->upPressed = true;
    else
        this->upPressed = false;

    if (GetKeyState(Input::Down) & 0x8000)
        this->downPressed = true;
    else
        this->downPressed = false;

    if (GetKeyState(Input::Left) & 0x8000)
        this->leftPressed = true;
    else
        this->leftPressed = false;

    if (GetKeyState(Input::Right) & 0x8000)
        this->rightPressed = true;
    else
        this->rightPressed = false;

}
```

# Game Engine Components

## Input

For each key, we apply a “hack”:
We check if the high-order bit is set
(1 << 15)
…and just set the properties of the Input class accordingly

```
#include "Input.h"
#include <Windows.h>
void Input::GetInput()
{
    if (GetKeyState(Input::Up) & 0x8000)
        this->upPressed = true;
    else
        this->upPressed = false;

    if (GetKeyState(Input::Down) & 0x8000)
        this->downPressed = true;
    else
        this->downPressed = false;

    if (GetKeyState(Input::Left) & 0x8000)
        this->leftPressed = true;
    else
        this->leftPressed = false;

    if (GetKeyState(Input::Right) & 0x8000)
        this->rightPressed = true;
    else
        this->rightPressed = false;

}
```

# Game Engine Components

## Game Loop

We can continue with our cleared game loop…

… now, we will use the *Input* class

```
int main()
{
    GameTimer timer;
    timer.initialize();
    while (true)
    {
        if (timer.checkFrame())
        {
            timer.onFrameBegin();

            // Do something
            CheckUserInput();
            ApplyGamePhysics();
            Render();

            timer.onFrameEnd();
        }

    }
}
```

# Game Engine Components

## Game Loop

We permanently query the input to so that keys can also be pressed between frames

```
input.GetInput();
if (timer.checkFrame())
{
    timer.onFrameBegin();

    if (input.leftPressed)
        character.MoveLeft();
    if (input.rightPressed)
        character.MoveRight();
    if (input.upPressed)
        character.Jump();
```

# Game Engine Components

## Game Loop

```
input.GetInput();
if (timer.checkFrame())
{
    timer.onFrameBegin();

    if (input.leftPressed)
        character.MoveLeft();
    if (input.rightPressed)
        character.MoveRight();
    if (input.upPressed)
        character.Jump();
```

If the abstracted buttons are pressed, we move a character (that has been instantiated before)

# Game Engine Components

## Game Loop

```
CheckUserInput();
ApplyGamePhysics();
Render();
```

The input system of the (simple) game engine is completed…




# Rendering

# Game Engine Components

## Rendering

In the main method, we instantiate and initialize the game timer, the renderer, and create a couple of physics objects with some properties…

```
//Global Variables
GameTimer timer;
timer.initialize();

RenderPlane2D renderer;

int framecounter = 0;

PhysicsObject fall(3, 29, 1, 1);
PhysicsObject throwUp(15, 0, 1, 1);
fall.name = "Fall";
fall.bump = true;
throwUp.name = "Up";
fall.collider = true;
fall.x = 10;
throwUp.x = 20;
throwUp.collider = true;

PhysicsObject block(0, 10, 1, 1);
block.name = "Block";
block.collider = true;
block.isRigidBody = false;
block.x = 25;
```

# Game Engine Components

## Rendering

We further instantiate a character and add all the objects to the world model of the engine

```
Character character;
character.name = "character";
character.width = 0.5;
character.height = 2;

WorldModel worldModel;
worldModel.addObject(fall);
worldModel.addObject(throwUp);
worldModel.addObject(block);
worldModel.addObject(character);
```

# Game Engine Components

## Finalization

This is our final, small and elegant game loop

- we check for input (permanently, not only when a frame is called)
- we set the corresponding timer methods to sync the global timeline
- we process input by forwarding it to the character
- we update the physics and game world with the step-function of the world model
- finally, we render the scene…

```
while (true)
{

    input.GetInput();
    if (timer.checkFrame())
    {
        timer.onFrameBegin();

        //Game Loop Code

        if (input.leftPressed)
            character.MoveLeft();
        if (input.rightPressed)
            character.MoveRight();
        if (input.upPressed)
            character.Jump();

        worldModel.Step(timer.deltaTimeSeconds);

        renderer.render();

        framecounter++;
        timer.onFrameEnd();
    }
}
```

# Game Engine Components

## Finalization

We can move around with our character, jump, and collect the physics objects which behave according to real Newtonian physics!



# Game Engine Components

## Our Game Engine

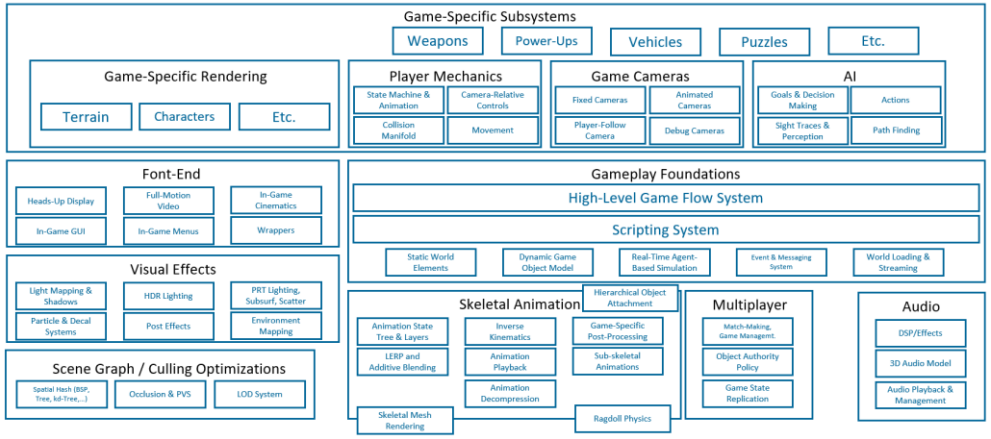
- **Relevant Improvement Suggestions**
  - Create a superclass for game objects so that positions are not bound to physical objects
  - Create a collider class to have collisions independent of physic objects
  - Create a class for the game/collision/physics world to make the population of the game independent of the renderer (using inheritance and polymorphism)
  - Hide potential private members of the individual classes and provide getter/setter
  - Abstract input from character movement and allow registering multiple characters for input
  - Abstract debug output in an interface so that all classes/components can add to the debug string
  - Create a graphical renderer instead of the command line

# Game Engine Architectures

## Our Game Engine

Our game engine has the same basic structure as a sophisticated “big engine” – now you know how this works!






```
// Called to bind functionality to input
void APlayerPawn::SetupPlayerInputComponent(UInputComponent* PlayerInputComponent)
{
    Super::SetupPlayerInputComponent(PlayerInputComponent);
}
```

```
000000B0: 65 6C 6C 6F 57 6F 72 6C:64 01 00 10 6A 61 76 61 | elloWorld☺ ►java
000000C0: 2F 6C 61 6E 67 2F 4F 62:6A 65 63 74 01 00 10 6A | /lang/Object☺ ►j
000000D0: 61 76 61 2F 6C 61 6E 67:2F 53 79 73 74 65 6D 01 | ava/lang/System☺
000000E0: 00 03 6F 75 74 01 00 15:4C 6A 61 76 61 2F 69 6F | ♥out☺ §Ljava/io
000000F0: 2F 50 72 69 6E 74 53 74:72 65 61 6D 3B 01 00 13 | /PrintStream;☺ !!
00000100: 6A 61 76 61 2F 69 6F 2F:50 72 69 6E 74 53 74 72 | java/io/PrintStr
00000110: 65 61 6D 01 00 07 70 72:69 6E 74 6C 6E 01 00 15 | eam☺ •println☺ §
```

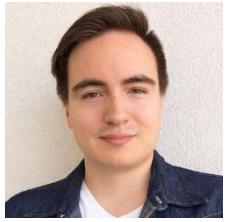
- **In these lectures, we have learned**
  - The concept of general machines (i.e., computers)
  - How languages like C++ make use of central machine components such as memory, pointers, and calculations
  - How we can create a software that uses these principles to simulate physics and interactive agents in real time (i.e., a game engine)
  - We have created a simple game engine ourselves and also used various concept in the Unreal Engine!

# Game Engine Architectures

**FH Oberösterreich**
University of Applied Sciences Upper Austria

**Digital Media Department**

Media Technology & Design
Digital Arts
Interactive Media



**Dr. Andreas Riegler**
Professor of Interactive Systems
Digital Media Department
E Andreas.Riegler@fh-hagenberg.at